

Assignment 4 – Distributed execution environment with mobile code and transparent migration

Stage 1: Runtime Environment for SimpleScript

Develop a runtime environment for the (intentionally simple) SimpleScript language with the following syntax and semantics:

<pre> Program = Tag {InstrLine}. InstrLine = [Label] Instr. Instr = SET '@'VarName VarVal ADD '@'VarName VarValI1 VarValI2 SUB '@'VarName VarValI1 VarValI2 MUL '@'VarName VarValI1 VarValI2 DIV '@'VarName VarValI1 VarValI2 MOD '@'VarName VarValI1 VarValI2 BGT VarValI1 VarValI2 Label BGE VarValI1 VarValI2 Label BLT VarValI1 VarValI2 Label BLE VarValI1 VarValI2 Label BEQ VarValI1 VarValI2 Label BRA Label OUT {VarVal} RD {VarVal '@'VarName} IN {VarVal '@'VarName} SLP VarValI PRN {VarVal} RET. Tag = #SIMPLESCRIPT VarVal = VarValI VarVals. VarValI = '\$'VarName IntVal. VarVals = '\$'VarName StrVal. VarName = Letter {Letter Digit}. Label = '#' Letter {Letter Digit}. IntVal = [-] Digit {Digit}. StrVal = " {Letter Digit} ". </pre>	<pre> VarName = VarVal VarName = VarValI1 + VarValI2 VarName = VarValI1 - VarValI2 VarName = VarValI1 * VarValI2 VarName = VarValI1 / VarValI2 VarName = VarValI1 % VarValI2 if (VarValI1 > VarValI2) goto Label if (VarValI1 >= VarValI2) goto Label if (VarValI1 < VarValI2) goto Label if (VarValI1 <= VarValI2) goto Label if (VarValI1 == VarValI2) goto Label goto Label add tuple with corresponding field read tuple with corresponding fields read & remove tuple with corresponding fields sleep(VarValI) print values {VarVal} followed by a new line return </pre>
--	--

SimpleScript programs have the form of assembly-like code. Each instruction is given in a separate line with its parts separated by one or more white space characters. The language supports integer and string values. Respective variables are dynamically declared/created when preceded by '@' and they are dereferenced (to get the current value) when preceded with '\$'. The program arguments are stored in prespecified variables \$arg<n>, where n is the argument number. By convention, \$arg0 is a string with the name of the program, while variable \$argc holds the number of available arguments.

Develop a SimpleScript interpreter, which takes as input the program name and arguments, and executes the code instruction by instruction. In case of a syntax error or an instruction with incompatible arguments, the interpreter must print a suitable error message and terminate the execution of the program. For convenience, the core logic of the interpreter can be implemented as a function that takes as input the program state (incl. the program counter) and returns after executing a single instruction. At this stage, the OUT/RD/IN instructions may not be supported (raising a syntax error) or be treated as no-ops.

Next, develop an environment which supports the command *run* <progrname> <arg1> ... <argN> for the execution of a SimpleScript program. Each such execution must be done conceptually via a separate “thread”. The context switching between such threads must be automatic under the control of the runtime environment. Note that the same program can be executed concurrently through different threads. The environment must also support the command *list* for printing the number and program name of each thread, and the command *kill* <thread id> for the termination of a specific thread. Furthermore, the PRN instruction must be implemented to automatically print as a preamble the identifier of the thread executing the program that invokes this instruction.

Stage 2: Shared Tuple Space

Extend the runtime environment to support the `OUT/RD/IN` instructions in the spirit of operations on a shared tuple space. These instructions take at least one argument, optionally followed by an arbitrary number of additional arguments, constituting a tuple expression. In the `OUT` instruction, each element of the tuple expression must be a concrete integer or string value (it can be a literal or a **dereferenced variable**). In the `RD/IN` instructions, some elements can correspond to concrete values serving as a tuple selection filter, **while other elements can be free or newly introduced variables (denoted by their name preceded by '@')** in which case they will be used to store the values of the respective fields of the selected tuple. The tuple space itself must be implemented as part of the runtime environment, using a suitable dynamic data structure.

Stage 3: Distributed Communication

Extend the runtime environment to support distributed communication between threads running on different runtime environments. More specifically, it should be possible for a `RD/IN` instruction of a local thread to be matched by a tuple that was produced via the `OUT` instruction of a remote thread, in a transparent way. To this end, the available runtime environments must discover and communicate with each other (to share/check each other's tuple contents) in a suitable way. For simplicity, the implementation does not have to ensure fairness and starvation freedom for the `RD/IN` instructions.

Stage 4: Thread Migration

Extend the runtime environment to support the live migration of threads between environments running as separate processes on the same or on different machines. To this end, the runtime environment must support an additional command *migrate* `<thread id> <ip addr> <port>`, specifying the thread to be migrated and the target runtime environment. Under the hood, the current environment must save the relevant state of the thread and transfer it (together with the program code) to the target environment, which, in turn, must restore the state and launch a new thread that will continue the execution from where it stopped. Note that the thread identifier must remain the same as in the previous runtime environment (thread identifiers must be globally unique). The implementation of the `OUT/RD/IN` instructions must be adapted to work properly under migration.

Stage 4: Load Balancing

Extend your implementation to automatically distribute threads among the available runtime environments so that each environment has roughly the same number of threads. Develop a suitable protocol for the necessary information exchange among the available environments and a simple heuristic for achieving the desired load balancing. Take care to avoid useless oscillations where one or more threads continuously migrate without improving the effective load balance in the system.

Further, extend the runtime environment to support the additional command *shutdown* for the smooth termination of the local environment. Upon shutdown, all locally running threads and the contents of the local tuple space should be migrated to other available runtime environments. If the local environment is the only available environment, the command should do nothing and print a corresponding message.

Optional extensions:

Starvation freedom. Ensure no thread will block on the `RD/IN` instructions indefinitely while other competing threads manage to successfully execute these instructions for compatible/matching tuples.

Deadlock detection A. Detect potential deadlocks, where all threads are blocked on an `RD/IN` instruction that cannot be satisfied based on the current contents of the tuple spaces. Note that this does not necessarily imply a true deadlock. This is because a new thread may start executing in the future and produce matching tuples via `OUT`.

Deadlock detection B. Detect true deadlocks, where tuple spaces have limited storage capacity (specified when starting the runtime environment), and all threads are blocked on an `RD/IN` instruction that cannot be satisfied based on the current contents of the tuple spaces, or on an `OUT` instruction that cannot be satisfied because all tuple spaces are full.

Checkpoint and restart. Support the checkpointing and deactivation of the runtime environment so that it is possible to restart/continue the execution of all local threads and restore the contents of the local tuple space in a transparent way when reactivating the environment at a later point in time. You may assume that the runtime environment will be restarted on the same host with the same contact information (IP address and ports).

You may implement this assignment using any programming language you wish.

Submission deadline (see instructions in eclass): **Saturday 23 May 2026, 22:00**